Lossless KV Cache Compression for LLM Serving: Exponent Coding in the Cache Tiers and in Attention

[TBD: authors; at least one non-academic author required by the CFP]

***Abstract*—LLM serving engines spill the attention key/value (KV) cache from GPU memory to host DRAM and NVMe so that agent and multi-turn requests skip recomputing shared prefixes. Two capacities then set time-to-first-token (TTFT) and goodput: whether the tiers still hold a session's prefix when it returns, and how many long requests the GPU KV pool runs at once. Lossy KV compression enlarges both but changes the values the model attends over. We present ExactKV, a *lossless* codec for BF16 and FP8 KV with GPU encoders and decoders at two integration points of vLLM and LMCache. In LMCache's cache server (tier mode) it stores every chunk as a variable-length record in DRAM and NVMe and restores it into vLLM's paged blocks, with no change to vLLM and no host synchronization on its load path. As a vLLM attention backend (pool mode) it keeps the GPU pool itself compressed behind a fused decode-attention kernel. A two-level 3-bit code for the sign and exponent stores BF16 KV at 0.70–0.72 of its bytes on nine models (FP8 KV at 0.90); the fused kernel reads 0.70 of the bytes and runs 1.14–1.26× faster per call than FlashAttention-3 (FA3) at the GPU's power cap, with FA3's error against an FP64 reference. On the AgentX benchmark (Qwen3-30B-A3B and GLM-4-9B, two H800s), compressed capacity serves as 1/0.716 = 1.4× raw capacity in both places. Counted in raw-equivalent bytes, compressed tiers follow raw LMCache's results, so the same hardware holds more sessions: with 400 GiB of DRAM, 480–640 GiB of NVMe give raw LMCache's results on 800–1200 GiB, 2.8–3.4× its goodput on the same tiers at LRU's cliff, and DRAM alone keeps 0.91 of prompt tokens cached for 40 sessions where raw keeps 0.45. When every request hits, a compressed set that fits DRAM cuts warm TTFT 1.6–1.7×. A compressed GPU pool performs as a raw pool of its token count: at 28 GiB per GPU it serves 1.57× tier mode's goodput; at 54 GiB it admits 12 requests with the queue raw has at 8. With capacity to spare the codec costs 0–2.2%. Tier mode's outputs are bit-identical to raw tiers; LMCache's lossy FP8 serializer changes 21 of 240 warm long-context answers.**

***Index Terms*—LLM serving, KV cache, lossless compression, tiered storage, GPU**

I. Introduction

Prefix reuse saves much of the work in production LLM serving: agent loops re-send a growing transcript every turn, and chat and retrieval-augmented (RAG) requests share history and documents [19], [21]. KV caches are large: Qwen3-30B-A3B stores 96 KiB per token, so a GPU that reserves 32 GiB for KV fits five 64K-token agent sessions. Engines such as vLLM [9] and SGLang [10] therefore spill KV blocks to host DRAM and NVMe through external tiers (LMCache [1], Mooncake [2], SGLang HiCache [48], CachedAttention [3], Strata [11]) and load them back on a prefix hit.

Four observations. Fig. 1 measures raw KV on our testbed. *Capacity* (a): agent sessions return in turn after long gaps, so

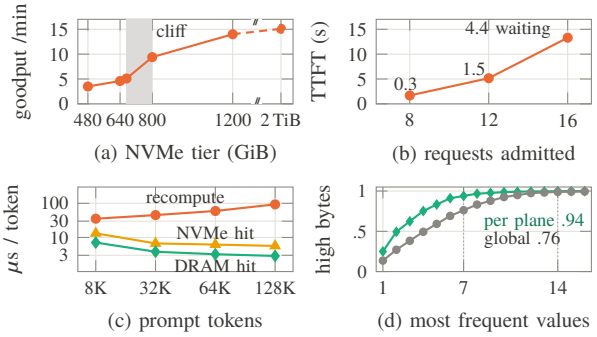


Fig. 1. Four observations on raw KV (Qwen3-30B-A3B, TP 2 on two H800s). (a) Capacity: AgentX goodput of raw LMCache (requests per minute with engine TTFT ≤2 s and ITL ≤40 ms; 64 sessions, 641 GiB of live KV, 400 GiB DRAM) by NVMe tier size. (b) Inference: with every prefix cached (2 TiB NVMe, 54 GiB of KV per GPU), mean engine TTFT by requests admitted at once; labels: requests waiting in vLLM. (c) Transfer: time per prompt token of a full hit loaded from DRAM or NVMe, and of recomputing the prompt. (d) Data: share of BF16 high bytes covered by the most frequent values, with one table per layer and K/V or one for the tensor (labels: at seven values); the low byte carries 7.97 of 8 bits.

LRU tiers that cannot hold the sessions' KV evict each prefix shortly before it is reused. With 64 AgentX sessions [20] (641 GiB of live KV on average, 755 at p90) and 400 GiB of DRAM, raw LMCache on 2 TiB of NVMe serves 0.94 of prompt tokens from a cache and 15.1 requests per minute within the target (engine TTFT ≤2 s, ITL ≤40 ms); on 640 GiB it caches 0.55 and serves 4.6. *Inference* (b): a 96K-token prompt holds 4.5 GiB of KV on each of two GPUs, so the engine's pool caps the requests in flight; with every prefix cached, admitting 16 instead of 8 leaves 4.4 waiting in vLLM and raises mean TTFT from 1.7 to 13 s. *Transfer* (c): a hit loaded from DRAM costs 3–7 μs per prompt token and one from NVMe 5.6–13, against 35–93 to recompute, so what matters is that the hit exists, not how fast it loads. *Data* (d): the low byte of BF16 KV is noise, but within one layer and one of K/V the high byte takes few values: seven cover 94% of scalars. A lossless code could thus store KV at about 0.7 of its bytes, and a compressed tier or pool would act as a raw one 1.4× larger, if storing, loading and attending over records cost nothing extra. Whether that holds, and what it buys, is what this paper measures.

Lossless vs lossy. Most KV compression is lossy: quantization (KIVI [22], FP8 KV), token dropping, or transform codecs

such as CacheGen [4] and KVTC [55] (up to 20× for KVTC). A lossy codec changes the stored values, so accuracy must be requalified per model and context length, and a request's values depend on the tier that served its prefix. ExactKV's code is lossless: with the codec in the tiers, outputs equal those of raw storage bit for bit (§VIII-E). The fused attention kernel sums in a different order than FlashAttention-3 (FA3) [45] and, like any backend change, is qualified once; its outputs still do not depend on where a prefix was stored. FP8 KV is itself lossy, but once an operator accepts it, ExactKV stores the FP8 bytes losslessly with a smaller gain (0.90 of FP8 in LMCache's DRAM on Qwen3-30B-A3B) that pays where FP8 KV fills DRAM.

Challenges. (R1) *Ratio:* the low byte of every BF16 scalar is noise, and FP8 has only a 4-bit exponent, so no order-0 per-plane code goes much below 0.67 of BF16 or 0.83 of FP8 and the format has little room for overhead. (R2) *Paged restore:* the engine's cache is scattered blocks, and decoding into a separate buffer costs a copy larger than the decode. (R3) *Off the critical path:* a record's length depends on its data, and learning it on the host must not delay a step or a load; GPU compressors (nvCOMP [30]) leave R2 and R3 to the caller. (R4) *Inside an existing tier:* the tier must keep each record at its length, or the savings buy PCIe bytes but no capacity, without the codec taking over its memory management, eviction or prefetch. (R5) *Attention over compressed KV* at FlashAttention's speed [45], with records allocated in the paged pool on the GPU.

Contributions. ExactKV runs with unmodified vLLM 0.30 in two places: inside the cache server of LMCache 0.5.6 [1], the KV-cache layer vLLM ships connectors for, and as an out-of-tree vLLM attention backend.

- **One lossless code for BF16 and FP8 KV** (§IV): a two-level 3-bit code for the high byte (ZipServ's has one level) with per-plane codebooks; tiles decode independently, straight into vLLM's paged blocks. It stores BF16 KV at 0.70–0.72 of raw on nine models and FP8 KV at 0.90 on Qwen3-30B-A3B; the concurrent ZipFusion's fixed 12-bit blocks store the same BF16 KV at 0.79 (Table II).
- **The codec inside LMCache** (§V): stores encode on a worker stream and reserve one exact-length object per record; loads decode in place on the transfer stream with no host synchronization; LMCache's L1/L2 management, eviction and prefetch are unchanged.
- **Attention over lossless variable-length records** (§VI): a fused decode-attention kernel decodes records laid out in its thread order into tensor-core operands; records are packed into vLLM's pages on the GPU, and decode steps never synchronize with the host, so they still replay as CUDA graphs.
- **Evaluation in the same two steps** (§VIII): tier capacity, then GPU capacity, on AgentX with LMCache deployed at its best (eager DRAM allocation and a 90% eviction watermark raise raw LMCache's throughput 6–7% over its defaults); kernel comparisons against nvCOMP, SplitZip, ExANS and FA3; costs, exactness and accuracy; lessons.

Results, in two steps. *Tiers* (§VIII-C): by raw-equivalent bytes compressed and raw LMCache agree within raw's run spread at the cliff and within 4% above it, so the codec on 480–640 GiB of NVMe gives raw's goodput on 800–1200; when every request hits, holding the compressed set in DRAM cuts warm TTFT 1.6–1.7× (1.3–1.4× when every hit comes from NVMe) with bit-identical outputs. Lossy alternatives store less but change outputs: vLLM's FP8 KV lowers RULER multi-key accuracy from 0.99 to 0.95, and the codec composes with it. *GPU pool* (§VIII-D): the compressed pool holds 1.31–1.34× the tokens; at 28 GiB per GPU it matches a raw pool enlarged to its token count and serves 1.57× tier mode's goodput; at 54 GiB it admits 12 requests with the queue the raw pool has at 8 and serves 13% more input tokens at raw's goodput, as on GLM-4-9B. The fused kernel's step is 1.3–7.6% shorter than FA3's, so the capacity is not paid back in decode time. Code: https://github.com/GentleCold/LMCache/tree/lossless-kv.

II. Background and Motivation

A. External KV tiers in vLLM

vLLM stores KV in a paged pool of 128-token blocks shared by all requests. A KV connector (`KVConnectorBase_V1`) has two halves. On admission the scheduler half reports how many prompt tokens beyond the GPU prefix cache are available externally; vLLM allocates blocks and, with asynchronous loading, parks the request until the worker half reports the load done. After each step the worker half may save new blocks. Placement is the connector's; the engine only sees block IDs.

LMCache's cache server. In its multiprocess mode, LMCache runs one cache server per node next to the engine. Each vLLM worker registers its paged KV cache by CUDA IPC handles, so the server moves blocks between the engine's GPU memory and its own tiers with its own kernels and streams: L_1 is a pinned DRAM pool, L_2 one or more adapters (here a native file adapter with O_DIRECT on NVMe). Stores land in L_1 and are written through to L_2 asynchronously; a lookup prefetches L_2-only chunks into L_1 and the load copies them from L_1 into the engine's blocks. A once-a-second loop evicts least-recently-used L_1 objects when occupancy passes a watermark. Every object is a fixed layout of the raw chunk.

B. Four questions about compressing KV

Fig. 1 answers four questions about compressing KV. *Capacity* (a): the cached share, and with it goodput, follows DRAM plus NVMe and changes abruptly where the tiers stop holding the sessions' KV, so where raw tiers fall short, 1.4× their capacity can decide between a hit and a recompute. *Inference* (b): the paged pool holds the KV of every running request, so once it binds, requests wait in vLLM with their prefixes cached, and only a compressed pool that attention reads in place adds requests in flight. *Transfer* (c): even from NVMe a load costs 0.37 of recomputing or less (a sixteenth at 128K), so a faster load matters less than whether the request hits, as for lossless paging and sharing [16]; fewer bytes speed loads only from NVMe when DRAM is small relative to the working set [8], [7], [14]. *Data* (d): §II-C. A codec must

therefore keep the capacity it saves: exact-length records in the tiers (R4), a compressed pool the kernel attends over in place (R5), and no load or step that waits for it (R2, R3).

C. The statistics of BF16 KV

A BF16 scalar is a sign/exponent byte and a mantissa byte. We dumped the KV of 4096 tokens of agent text from Qwen3-30B-A3B (201 M scalars). The low byte carries 7.97 of 8 bits; it is incompressible. The high byte carries 3.57 bits over the whole tensor, but only 2.85 bits within one layer and one of K/V (a *plane*), because each plane concentrates on a few values: with tables fitted per plane, the seven most frequent values cover 93.9% of scalars, the next seven 5.9%, and 0.19% fall outside fourteen. The order-0 bound is therefore 0.721 of raw with one global table and 0.676 with one table per plane; conditioning on the channel (head and dimension) lowers it to 0.659. A byte-shuffling zstd reaches 0.687 per chunk on the CPU. The per-plane bound thus caps a per-plane order-0 code's capacity gain at 1.48×. FP8 E4M3 KV is one byte per scalar: sign, four exponent bits and three mantissa bits. Its per-plane order-0 bound is 6.64 bits per byte (0.830), so the same kind of code can save at most a sixth of FP8's bytes.

D. Why not an existing codec

nvCOMP's codecs treat KV as bytes, need contiguous input and report lengths through the host. Weight codecs [31], [32], [35] compress offline; ZipServ's code is the nearest in form: a 3-bit code per BF16 exponent over a window of seven values, with full-precision fallback, decoded into tensor-core operands inside GEMM. Applied per plane to our KV, its high part costs 3.40 bits against 3.20 for ours, so with the same tile index it would store 0.728 of raw against ExactKV's 0.716 (Table II). Unlike weights, KV is encoded online on the serving GPU, restored into paged blocks and decoded inside attention. SplitZip [5] codes BF16 exponents with 4-bit symbols for prefill–decode transfer, with no paged destination, tier or record lifecycle. A codec in front of an existing tier does not add capacity by itself: tier stores size their objects for raw chunks, so compressed records save PCIe bytes but no capacity until the store keeps a length per record, which is what ExactKV adds to LMCache (§V).

III. System Overview

Two modes. ExactKV has two integration points that share the code of §IV. In *tier mode* (the codec in the tiers), vLLM keeps raw KV and its stock FA3 backend; LMCache's cache server encodes each chunk it stores and decodes each chunk it loads, so DRAM and NVMe hold compressed records and vLLM's outputs are bit-identical to raw storage. Tier mode needs no change to vLLM and leaves LMCache's tier management in place (§V). In *pool mode* (the codec in attention), a vLLM attention backend keeps the engine's own pool compressed and attends over it in place (§VI); it adds GPU capacity, which tier mode cannot. Both use the same 3-bit primary exponent code; escape levels and record layout differ, as pool mode lays its records out for the attention kernel (§VI-A), so in pool mode the cache server moves each closed block as an opaque blob and runs no codec (§VI-E).

Units. A *chunk* is the KV of one 128-token block across all layers: 12 MiB raw for Qwen3-30B-A3B in BF16, 6 MiB per rank at TP 2. LMCache keys chunks by a rolling hash of the token prefix. A chunk is split into *planes* (the K or V of one layer) and each plane into 256-scalar *tiles*. The codec turns a chunk into one *record*; LMCache stores the record as one object in L_1 and L_2.

Store and load paths. After the step that completes a chunk, vLLM's connector asks the cache server to store it. The server encodes the chunk straight from the engine's blocks into a slot of a GPU store pool and returns; a store worker of its own then reserves an L_1 object of each record's length, copies the record into it and commits it (A, B); LMCache writes the object through to L_2 (C). On a lookup, LMCache returns the longest prefix it holds and prefetches L_2-only chunks into L_1 (1, 2). The load copies the records into GPU record buffers on a stream of its own and decodes them straight into the engine's blocks (3, 4). The forward pass never waits for a store, and no load path step reads a length back to the host.

IV. Paged-Native Lossless Codec

A. Record format

Fig. 3 shows a coded tile. The low byte of each scalar is stored verbatim. The high byte is looked up in a static per-plane codebook of 14 values: a 3-bit primary symbol selects one of the seven most frequent values; code 7 escapes to a 3-bit secondary symbol that selects one of the next seven; secondary code 7 escapes to a raw byte. Within a plane, the low bytes, primary symbols, secondary symbols and raw bytes form four streams. Two persisted prefix tables give, for every tile, the count of secondary and raw symbols before it, so a decoder can start at any tile without scanning its predecessors (R2). Each chunk becomes one record: a 4 KiB-aligned header with a version, a mode (packed or raw) and one descriptor per plane giving the offsets of its streams, followed by the plane payloads and padding to 4 KiB for O_DIRECT IO. If a chunk would not fit in its raw envelope, it is stored in raw mode, so the worst case equals raw storage.

The format spends $8 + 3 + 3p_e + 8p_r + 0.25$ bits per scalar, where p_e and p_r are the secondary and raw escape rates and 0.25 bits are tile prefixes. With the generic codebook on dense models, $p_e = 0.147$ and $p_r = 0.0094$, which is 11.77 bits or 0.736 of raw. With calibrated codebooks on Qwen3-30B-A3B, $p_e = 0.061$ and $p_r = 0.0019$, which is 11.45 bits or 0.716, the ratio we measure.

Why 3+3 bits. On the dump of §II-C the high byte costs 3.49 bits with one 3-bit level and raw escapes, 3.40 with ZipServ's window of seven exponents [35], 4.00 with one 4-bit level (as in SplitZip [5]), 3.61 with ALP's exceptions [42], 3.24 for 2+3 or 3+4 and 3.20 for 3+3, against an order-0 entropy of 2.85. A variable-length code would buy at most 3% more capacity but serialize decoding within a tile.

Codebooks are fixed at startup: a generic table, or one calibrated offline per model from one forward pass (1024

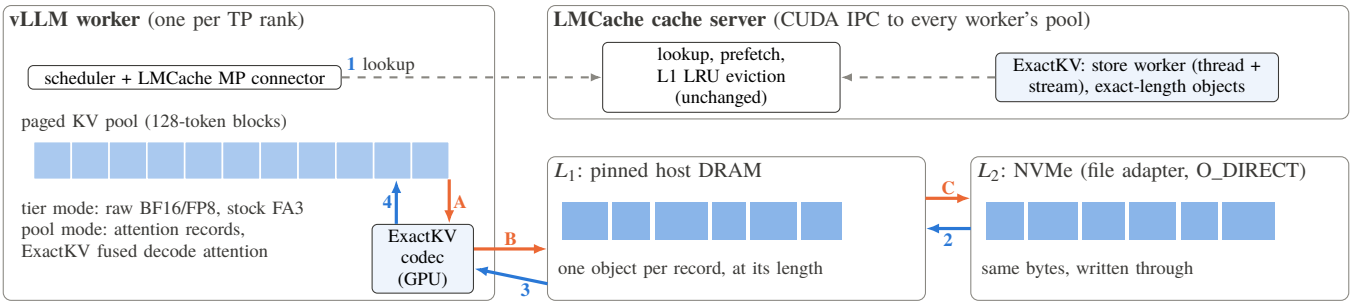


Fig. 2. ExactKV in vLLM and LMCache. Load path (blue numerals; dashed: control): 1 lookup in the cache server, which prefetches L_2-only chunks; 2 L_2 to L_1; 3 copy of the records to the GPU; 4 decode straight into the engine's blocks (tier mode) or import of attention records (pool mode); Orange, store path: A encode from the engine's blocks into the store pool after the producing step; B the store worker reserves one L_1 object per record and copies it out; C LMCache's write-through to L_2. Kernels run on the engine's GPU, launched by the server through CUDA IPC. Only compressed bytes cross PCIe and reach DRAM and NVMe.

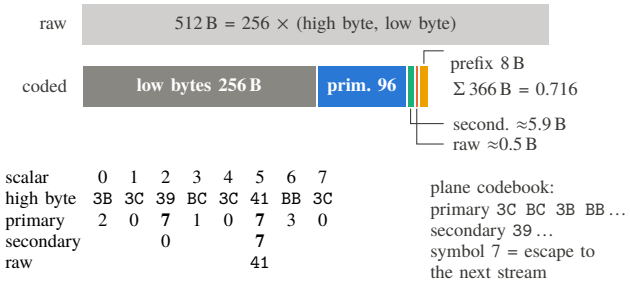


Fig. 3. One tile of Qwen3-30B-A3B KV, mean sizes with calibrated codebooks. Low bytes are stored verbatim; high bytes become a 3-bit primary symbol, escapes (7) a 3-bit secondary symbol, and secondary escapes a raw byte. Bottom: one lane's eight scalars. Tile prefixes locate each tile's secondary and raw segments, so every tile decodes independently.

TABLE I
STORED RATIO OF BF16 KV BY MODEL: 4096 TOKENS OF AGENT TEXT, CODEBOOKS CALIBRATED ON THE FIRST 1024 TOKENS AND MEASURED ON THE REST. TIER RECORDS WITH THE GENERIC AND THE CALIBRATED CODEBOOK, THE PER-PLANE ORDER-0 BOUND, ATTENTION RECORDS AND THE MEAN SIZE OF THEIR VARIABLE PART. $L\times H$: LAYERS × KV HEADS.

		tier records			attention rec.	
Model	$L\times H$	generic	calib.	bound	ratio	var. B
Qwen3-4B	36×8	.736	.716	.676	.705	562
Qwen3-8B	36×8	.739	.716	.676	.705	561
Qwen3-32B	64×8	.738	.715	.674	.703	515
Qwen3-30B-A3B	48×4	.771	.716	.676	.704	550
Qwen2.5-72B	80×8	.729	.714	.672	.702	484
Llama-3-8B	32×8	.739	.714	.672	.702	479
Llama-3.1-8B	32×8	.736	.713	.672	.701	454
Llama-3.1-70B	80×8	.747	.712	.669	.700	395
GLM-4-9B-1M	40×4	.737	.714	.673	.702	459

tokens suffice, Table I) and bound to the model by a digest. Calibration lowers Qwen3-30B-A3B's ratio from 0.771 to 0.716; tables calibrated on agent text and on the benchmark's prompts differ by 0.0003. On nine Qwen, Llama and GLM models, 4B to 72B, dense and mixture-of-experts (MoE) (Table I), calibrated tier records store 0.712–0.716 of raw, 0.04 above the order-0 bound, and attention records (§VI-A) 0.70.

B. Fused decoder

One warp decodes one tile; each lane owns eight consecutive scalars. A lane spreads its 24-bit primary word to one nibble per symbol. An escape is the symbol 7 (all three bits set), so intersecting the nibbles with their one- and two-bit shifts marks escapes and a population count gives the lane's escape count without a per-scalar ballot. Two warp-wide exclusive scans plus the tile prefix give the lane's offsets into the secondary and raw streams. In Fig. 3 the lane marks scalars 2 and 5, takes two secondary symbols (0 and 7), and, since the second escapes again, one raw byte (41).

High bytes come from byte-permute (PRMT) instructions. Codebooks are repacked to 16 bytes per plane (primary entries in bytes 0–6, secondary in 8–14, escape entry zero), so a symbol nibble is directly a PRMT selector and one PRMT pair resolves a level for four scalars; one more PRMT, indexed by escape ranks, gathers escaped bytes, and an OR merges them because escape entries decode to zero. Decoding is branchless, since escapes are scattered across lanes. Values stay in registers and go out as 128-bit stores directly into each destination block, reusing the registers when one chunk fills several blocks. A thread block (CTA) decodes 32 groups of eight tiles and prefetches the next group's words.

C. Fused encoder

The encoder runs in three launches with no host synchronization between them: a *count* stage reads the KV straight from the engine's paged blocks and scans per-tile escape counts, a single-warp *layout* stage assigns record offsets and decides raw fallback, and a *compact* stage writes the streams, merging secondary bits of different lanes with atomic adds on disjoint bit ranges. It runs on a store thread and stream behind the producer event of the step that wrote the KV; the thread synchronizes once per batch to read the stored lengths (R3), so the engine thread neither encodes nor waits.

D. Robustness

Every global write address of the decoder comes from host-validated metadata; record bytes only choose values, and reads they address are clamped to the staging extent, so a corrupt

record restores wrong values into its own blocks and nothing else (checked by a GPU fuzz test that flips bits in descriptors, prefix tables and payloads). The decoder waits on a model-stream event before writing blocks whose previous owner may still be read.

V. The Codec Inside LMCache

A tier's manager decides what to keep; ExactKV changes only what an object holds and how it moves between the GPU and L_1. LMCache's L_1 and L_2 management, LRU eviction, write-through and prefetch stay as they are. Three things change: an object's size, who produces it, and how a load turns it back into blocks. All of it is enabled by one server flag and off by default, which leaves LMCache's raw path untouched.

A. Exact-length objects

LMCache allocates every object of a registered cache from one layout, the raw chunk's. With the codec, the cache registers a flat byte layout whose size only bounds a record (its raw envelope), and each record reserves an L_1 object of its own length from the pinned pool. The file adapter writes the object as it is; records are padded to 4 KiB, so O_DIRECT still applies. A prefetch from L_2 needs an L_1 buffer of the stored size, which only L_2 knows: the adapter records each object's size when its store completes, and the prefetch controller reserves byte-layout objects at those sizes, one reservation per distinct size. Without this, a compressed tier saves PCIe bytes but no capacity (§II). These changes add about 160 lines to LMCache's storage layer and leave typed layouts as they were.

B. Store path

A record's length is known only after encoding, and reading it synchronizes the GPU (R3). LMCache serves all stores and loads of one GPU on one handler thread, in order, so a synchronization in the store would delay every load queued behind it. The handler therefore only launches: it skips chunks already resident in L_1 (which LMCache would refuse to overwrite) and, for each batch of chunks (eight by default), encodes the engine's blocks straight into one slot of a pool of GPU buffers, records an event and returns; the engine may reuse its blocks once that event passes. A store worker, with its own thread and stream, waits on the event, reads the batch's lengths with one synchronization, reserves one L_1 object per record (records of equal length share a reservation), copies each record to its object and commits the batch. Encoding from the blocks replaces a gather into the pool, a copy into the codec's input and a copy out of its output: a batch costs five kernel launches instead of 22 and two passes over the raw bytes fewer. A full pool blocks the handler, so stores slow down rather than being dropped when encoding falls behind. A per-request tally reports the stored count and bytes once every batch has committed, so LMCache's accounting is unchanged.

C. Load path

A load names a prefix of chunks whose objects LMCache has pinned in L_1. Per batch, ExactKV copies each record into one of the codec's record buffers and decodes the batch straight into the engine's blocks (§IV-B); chunks wholly inside the engine's own prefix-cache hit are skipped. Batches with a raw record or a partly shared leading chunk decode into a scratch and are scattered with LMCache's transfer kernel, which handles vLLM's layouts. Record lengths come from the host-side object sizes, and the decoder's launch metadata depends only on the batch's length pattern, so it is uploaded once and reused: the load enqueues copies and kernels and never waits for the GPU. The copies run on a stream of their own and rotate through four sets of record buffers; each decode waits for its set's copies and each refill for that set's last decode. Copies of later batches thus run ahead while earlier batches decode, and the decodes queue back to back. One server thread handles every store and load of a GPU, so any wait there stalls all requests behind it (§IX).

Layouts and parallelism. vLLM 0.30's FA3 cache interleaves K and V per head (`[blocks, tokens, heads, 2×head]`); the codec views a block's K and V as separate planes by stride, so codebooks stay per (layer, K/V). Under tensor parallelism every rank registers its own cache and LMCache keys objects per rank; each rank's records have their own lengths. BF16 caches and vLLM's FP8 E4M3 caches (stored as bytes) use the same code with their own codebooks; an FP8 scalar's three mantissa bits are stored verbatim and its sign and exponent are coded (0.90 of raw; bound 0.83).

VI. Compressed KV in GPU Memory

Tier mode saves capacity outside the GPU, but the engine's pool, which decides how many long requests run at once, still holds raw KV. Pool mode keeps the pool itself compressed, through an attention backend registered with vLLM's plugin entry point (`--attention-backend CUSTOM`), without changes to vLLM's source. vLLM still allocates blocks, builds block tables and schedules requests, but each block's per-layer page holds *attention records*.

A. FragPack attention records

A *FragPack* record, named for storing scalars in fragment order, is the K or V of one block, layer and KV head: 128 tokens by 128 dimensions, 32 KiB raw. High bytes use the 3-bit primary code of tier records with a per-(layer, K/V, head) table of seven values; an escape takes a 4-bit nibble selecting one of 15 further values, or 15 for a raw scalar. The record differs from a tier record in scalar order: the decode kernel (§VI-D) runs 256 threads per record, each consuming 64 scalars as operand fragments of tensor-core matrix multiply-accumulate (MMA), and the record stores scalars in exactly that order, so each thread reads its own bytes with 16-byte loads and no index arithmetic. A fixed *dense part* of 22 528 B holds the low bytes and primary symbols, chunk-major so a warp load covers 512 contiguous bytes; a *variable part* of at most 2 KiB holds a 48-byte header of per-warp escape and raw prefix counts, the escape nibbles and the raw scalars. With calibrated tables the variable part averages 550 B on agent KV,

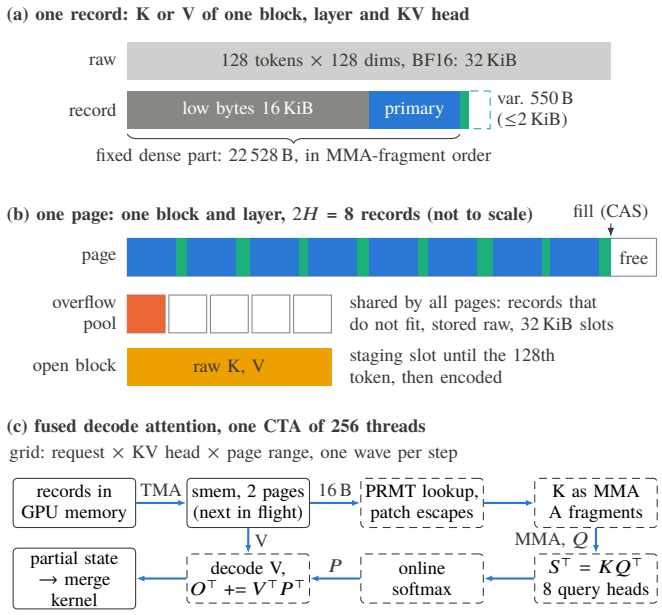


Fig. 4. The compressed GPU pool. (a) An attention record to scale: the dense part has a fixed size; the variable part holds a header of per-warp counts, escape nibbles and raw scalars. (b) Records pack into the page in encode order (blue: dense, green: variable); a record that does not fit spills raw to an overflow slot (illustrative: on agent KV none spills); per-layer tables hold every record's address and length. (c) Tensor Memory Accelerator (TMA) copies, issued once per CTA, bring each page's records into shared memory a page ahead; each thread decodes its 64 scalars into tensor-core operand fragments (dashed: registers only, so decoded KV never reaches GPU memory); a final kernel merges the page ranges' partial softmax states.

so a record is 0.70 of raw (Table I: 395–562 B on nine grouped-query attention (GQA) models). A record whose variable part exceeds 2 KiB, or with more than 32 raw scalars in a warp, is stored raw and read by the same kernel. FP8 records store mantissas verbatim and code sign and exponent less a per-channel offset (added back on decode), so channels at different scales share the head's primary values; offsets are fitted for K over positions up to 256K as RoPE rotates its channel pairs. On held-out agent KV at positions up to 240K the records are 0.84 of raw.

B. Pages and packing

A block's page for one layer is sized for its $2H$ records at an average variable part of 1280 B: 190 464 B for $H = 4$ against 262 144 B raw, so vLLM's KV budget yields 1.376× as many blocks (1.31–1.34× after the backend's buffers); the backend reports this size through its KV-spec hook. Records pack into the page in encode order (Fig. 4). The encoder runs one CTA per record, sizes the variable part v with block-wide scans and claims $22528 + v$ bytes by a compare-and-swap on the page's fill counter; a failed claim leaves the counter unchanged, so a smaller record of the block can still fit, and a record that does not fit goes raw to an overflow pool of 32 KiB slots, a bitmap allocated with atomic OR so encodes and imports on different streams can allocate concurrently. On agent KV, a 1280 B share spilled no record and a 1024 B share about 2%, each costing 32 KiB; fixed 24 KiB record slots (2 KiB for the variable part) would give 1.33×, against 1.38× with shared pages and 1.42× for an ideal heap. Per-layer tables give every record's address and variable length, so kernels never compute positions inside a page.

C. Per-step state machine

Only complete blocks are encoded; a block still filling keeps raw records in a staging slot. Before each step the backend releases the staging of blocks no request still writes, claims staging for blocks that receive their first token, and lists those that receive their 128th; each layer writes new K and V into staging, attends through the tables and encodes the listed blocks. A running request has at most one open block, so one slot per schedulable sequence suffices. vLLM does not tell backends when blocks are freed, so the plugin mirrors each request's block list from the scheduler output and treats as open exactly the block holding its first uncomputed token (a superset exhausted staging at 32 requests). Every launch depends only on the padded batch shape, so decode-only steps replay as full CUDA graphs.

D. Fused decode attention

Decode is split along the context (Fig. 4c): a CTA of 256 threads handles one request, KV head and page range. Once per step, the device picks one range length for all requests, the shortest whose CTAs fit in one wave, so each request gets CTAs in proportion to its context; equal ranges per request let the longest of eight agent contexts gate the step at 1.4–1.8× FlashAttention-3's time. TMA copies each page's records (dense part plus actual variable length) a page ahead. Each thread decodes its 64 K scalars with byte-permute lookups into a register-resident table, patching escapes located by a warp scan over the header's prefix counts, into the A fragments of an `m16n8k16` MMA; computing $S^\top = KQ^\top$ lets a GQA group's eight query heads fill $n = 8$. The kernel reads about 0.70 of the raw bytes. Raw records skip the decode step, with bitwise identical output. Prefill is compute-bound and runs FlashAttention-3 unchanged on a BF16 workspace into which the context pages are first decoded; fusing that step could save at most its 3% (§VIII-B).

E. Blobs in the external tiers

With attention records in the pool, the tiers need no codec. An export kernel copies a complete block's records of all layers ($2H$ per layer) into one *blob*: a 4 KiB-aligned head (version, geometry, a digest of the code tables, every record's variable length) followed by the records. An import kernel validates the head, packs the records into the destination page and only then swaps the record tables, so a failed layer keeps a consistent table; any failure marks the batch invalid and vLLM recomputes it. GPU, DRAM and NVMe then hold the same lossless bytes.

The cache server, not the worker, launches both kernels. The backend announces its page geometry through LMCache's layout hints when vLLM registers the KV cache, and shares its record tables over CUDA IPC before the first store or load.

On a store, the server exports the request's closed blocks into the store worker's fixed-size GPU pool on its own stream and returns; the worker reads the blob lengths back and commits one exact-length object per block (§V-B), so the engine's thread never waits on an export. A block still open or already freed fails its export and is simply not stored. On a load, the server copies the blobs into a staging buffer, imports them into the destination blocks and synchronizes once per request; a failure fails the load and vLLM recomputes. Because an LMCache chunk must then be exactly one 128-token vLLM block, pool mode fixes LMCache's chunk size to the block size.

VII. Implementation

ExactKV is a branch of LMCache 0.5.6 that runs with unmodified vLLM 0.30: the codec package (4.9 kLoC of Python and TileLang kernels), its integration into the cache server (1.0 kLoC, §V) and the attention backend (2.6 kLoC of Python, 1.8 k of CUDA C++), with 2.3 kLoC of tests. Codec on or off, codebooks and the store batch are fixed at server startup; the raw path is LMCache's own.

The attention backend is loaded as a vLLM plugin. Its 1.3 GiB of GPU memory (overflow pool, staging, prefill workspace) is charged to it in every AgentX comparison. Head size, GQA group and block size are compile-time parameters; we build for BF16 and FP8 KV, 128-dimension heads, GQA groups of up to 8 (smaller ones padded) and 128-token blocks.

VIII. Evaluation

Q1 compares the kernels with existing codecs and FlashAttention-3. The end-to-end evaluation then follows the two capacities: **Q2** (step 1) asks what compressed tiers buy on AgentX and when every request hits (tier mode); **Q3** (step 2) what a compressed GPU pool buys once the tiers hold every working set (pool mode); **Q4** what compression costs and what exactly is lossless.

A. Setup

Hardware. Two 48-core Xeon Platinum 8558, 1 TiB DRAM, four 3.84 TB Memblaze P6531 NVMe drives in one btrfs volume (single data profile). The server is a shared lab machine; we use two of its H800 SXM modules (80 GB, 700 W), which this chassis connects over PCIe without NVLink. No other job used the two GPUs during a run. **Models.** Qwen3-30B-A3B-Instruct, a mixture-of-experts (MoE) model with 3B active parameters (FP8 weights, BF16 KV, 96 KiB per token), at TP 2, unless noted; GLM-4-9B-Chat-1M, a dense model of another family (40 layers, 4 KV heads, 80 KiB per token, BF16 weights; 1.42M tokens in 54 GiB of KV per GPU), at TP 2 as a second AgentX model; Qwen3-8B and Llama-3.1-8B (Hermes-3 weights [46]) for kernel and ratio generality. vLLM 0.30 with its V2 model runner, asynchronous scheduling, asynchronous KV loading, 8192-token chunked prefill and 128-token blocks.

LMCache deployment. Every LMCache arm runs LMCache 0.5.6's cache server (multiprocess mode, LMCache-driven transfers), raw or with the codec, and differs from the others only in the flags we name. L_1 is 400 GiB of pinned DRAM, the GPU-proportional share of host DRAM that AgentX's harness grants two of our five GPUs, allocated eagerly (no lazy growth). L_2 is LMCache's native file adapter with O_DIRECT and 16 IO workers on a directory with copy-on-write disabled (on btrfs it cut sustained writes from 12.8 to 4.97 GB/s); on its own benchmark it loads 3 MiB objects (deployed: 6 MiB raw, 4.3 coded) at 6.6–7.4 GB/s and stores at 4.6 GB/s. Buffered IO would load at 47 GB/s from the page cache, a DRAM tier outside the budget, so we do not use it. L_1 evicts LRU objects once it is 90% full, 5% of its keys at a time; LMCache's defaults (80% and 20%) would leave a fifth of DRAM unused between its once-a-second eviction passes. A run starts only after the server reports its whole L_1 pinned and every L_2 adapter healthy (2–3 min for 400 GiB), so no arm loses capacity to initialization. **Baselines.** LMCache raw with the same budgets; LMCache with its FP8 L_2 serializer (lossy); vLLM with FP8 KV, which halves every level and changes the values, with raw LMCache tiers. We do not run Mooncake or SGLang HiCache: both keep raw KV and LRU-like tiers, so we expect their cliff near the same raw working set and the codec to move it the same way. CacheGen and TurboQuant are lossy, and CacheGen is not in LMCache's multiprocess server (§XI).

TABLE II
Codecs on one H800 SXM and the same Qwen3-30B-A3B KV (3.2 GB, 256 chunks per call), raw-equivalent GB/s. "Paged" restores into scattered blocks of a paged pool; "contig." stops at the codec's own output buffer.

Codec	ratio	enc	paged	contig.
Device copy (raw)	1.000	–	–	1523
Raw gather to paged	1.000	–	547	–
ExactKV	**0.716**	**528**	**1229**	–
SplitZip [5]	0.768	326	376	1264
ExANS [44]†	0.698	7	186	–
nvCOMP ANS	0.725	113	181	276
nvCOMP Bitcomp	0.876	116	152	212
nvCOMP LZ4	1.000	9	146	202
ZipServ code‡	0.728	–	–	–
ZipFusion blocks§	0.79	–	–	–

†Gathers and scatters 16 KiB pages itself; we batch its header readback into one copy (78 GB/s as shipped). ‡ZipServ's TCA-TBE code [35] computed offline, window per plane, with ExactKV's tile index (0.713 without it); ZipServ ships a weight compressor and GEMM only. §ZipFusion's format [33] computed offline: 512-value blocks with a 12-bit code, raw when a block spans more than 16 exponents (K 0.81, V 0.77); its kernel targets FlashInfer and has no tier path.

B. Q1: Kernel microbenchmarks

Table II compares codecs on the same card and KV, every restore checked byte-exact. Because ExactKV writes restored values from registers into the destination blocks, it restores into a paged cache at 81% of a contiguous D2D copy, while reading only 0.716 of the bytes; the raw-gather row (PyTorch's `index_copy_` of whole block rows, 256 blocks per call, not a tuned gather kernel) is 2.2× slower. A contiguous codec must decode and then scatter: SplitZip's decode reaches 83% of a D2D copy, and ExactKV is 3.3× faster than SplitZip plus

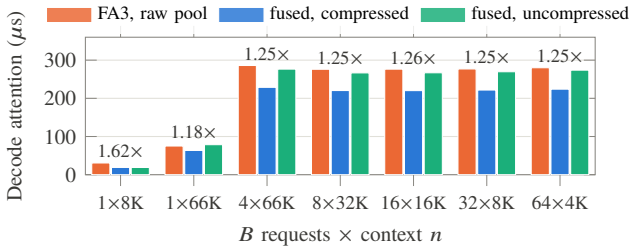


Fig. 5. Fused decode attention on one layer of Qwen3-30B-A3B KV (4 KV heads, 2112 blocks, one H800 SXM, back to back; at the 700 W cap except 1×8K): FlashAttention-3 over vLLM's raw pool, and the fused kernel over FragPack records, compressed (0.70 of the bytes) or with every record stored uncompressed (same kernel, decode skipped). Labels: FA3 time over fused compressed time.

our gather and still 1.8× faster if the scatter ran at copy speed, while storing 7% fewer bytes. ExANS stores 2.5% fewer bytes (0.698, between the global and per-plane order-0 bounds of §II-C), but restores 6.6× and encodes 75× slower.

Fused decode attention. Fig. 5 interleaves the kernels in one process, each called back to back. Batches hold the card at its 700 W cap, where time follows energy: FA3 takes 1.25–1.26× the fused kernel's time and energy (1.14–1.17× for a TP 2 rank's two heads), and raw-stored records take 1.21–1.22× longer, so reading 0.70 of the bytes pays for decoding. Followed by a memory-bound 320 μs GEMM sharing its clock, each call still saves 52–56 μs; Q2's BF16 fused-pool arms clock 7–20% higher at the same mean power (687 W, under the 700 W cap), and every FP8 KV arm with FA3 we logged holds the cap. One 8K context stays below the cap: its 1.6× (FA3 caps it at 32 splits) ties in the step, and idle-gapped batches take 0.95–1.14× FA3's time. Prefill over the decoded workspace costs 1.1–2.9% more than FA3 (2K–8K chunks).

C. Q2: Step 1, tier capacity on AgentX

We run AgentX [20] with its official harness (agentx-harness 1.0.7, dataset `cc_traces_weka_062126_256k`, seed 42). It replays the 393 Claude Code sessions of cc-traces [47] as session trees with subagents and recorded think times. Each client joins a session 25–75% of the way in, sends primers that rebuild its live contexts and ten warmup requests; all metrics cover only the profiling window that follows. Below the cliff the cached share still falls inside it (raw on 640 GiB: 0.88 to 0.17 by 5-minute bin, Fig. 7b), so window averages favor those arms. A token-0 marker, required by the AgentX scenario, keeps sessions from sharing a prefix, so two clients replaying the same trace session do not hit each other's KV. The harness caps DRAM offload at the GPU-proportional share of host DRAM, 400 GiB for 2 of our 5 GPUs. Prompts reach 254K tokens (p50 89K), so we serve Qwen3-30B-A3B with TP 2 on two H800s and fill each GPU: 14.6 GiB of weights and 54 GiB of KV (1.18 M tokens). The attention backend charges its 1.3 GiB to the same 54 GiB; the cache server's CUDA context and the codec's GPU buffers (1.0 GiB per GPU with the codec) live outside vLLM's budget in every LMCache arm.

Admission. The harness's concurrency counts live root sessions; subagent requests take no slot. Open loop, it ties the live working set to the requests in flight: at 16 clients every external tier holds the working set (0.95 cached), yet vLLM queues each request for 19–23 s. To grow the working set past DRAM without overloading the engine, a 140-line FIFO proxy admits S requests at a time, as `vllm bench serve --max-concurrency` does on the client side. Of S = 8 and 12, 8 is the larger value at which the stock pool keeps vLLM's waiting queue near empty when the tiers hold every working set (P = 32 and 64; S = 12 leaves 1.5–1.7 waiting), which favors the stock pool (§VIII-D). At S = 8 and 54 GiB vLLM's waiting queue averages 0.1–0.6 requests. TTFT as the harness measures it is then mostly the proxy wait (over 90% at P = 48), so we report engine TTFT (minus the proxy wait, which the admission cap sets) and the share of requests whose engine TTFT is at most 2 s (a cached 90K-token prompt starts in well under 2 s, its recompute takes several); the proxy is the same FIFO for every arm. Other users' jobs could use the host's CPUs and other GPUs; repeated runs show the spread.

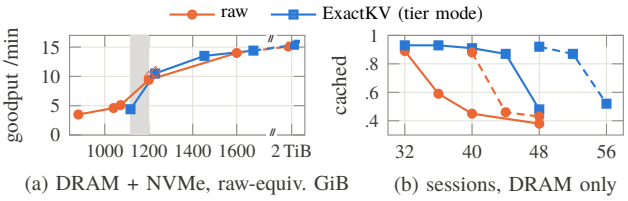
Sizing. With admission fixed, the number of live sessions P sets the reused working set (a session's KV is live from its first to its last request): mean 306, 380, 489, 584 and 641 GiB raw at P = 32 to 64 in steps of 8 (p90 755 GiB at 64). Below the cliff, slower arms finish shorter prompts in the window (raw on 640 GiB: mean 78.8K tokens, tier mode 84.1K), so we give request rates next to token rates; at $P \geq 48$ the loop saturates the GPU pair. Most runs use P = 64 with 400 GiB of DRAM and vary the NVMe tier from 400 GiB to 2 TiB, which holds every set. Each tier runs its own LRU (a DRAM hit does not refresh NVMe), and the cliff follows DRAM plus NVMe, not either tier alone (§X). Deployments with more sessions per GPU, more KV per token (Llama-3.1-70B: 320 KiB, 3.3× ours) or several engines sharing a node's drives reach these ratios with larger tiers. The profiling window is 1800 s instead of 3600; repeated runs differ by 1.1% (raw, 2 TiB), 9% (raw, 640 GiB: 12.2, 13.3 k), 0.2% (tier mode, 640 GiB), 13.5% (raw, 800 GiB), 6% (tier mode, 480 GiB), 2.6% (raw, DRAM only, P = 40), 35% and 2.5% (raw and tier mode, FP8 KV, DRAM only) and 2% (28 GiB bound). Requests without a first token by the window's end are not counted, which favors slower arms. The profiling window starts when the harness's warmup ends, and every run goes to the end of its window (warmup: 10–28 minutes for arms that hold the working set, 88–99 for those at the cliff whose NVMe holds at most 672 raw-equivalent GiB, 42–143 below it).

Capacity equivalence. Table III. With NVMe holding every working set, tier mode keeps raw LMCache's cached share (0.94), ITL and goodput (15.4 vs 15.1 per minute) and costs 2.2% of its throughput, against 1.1% between raw runs; its median TTFT is 12% lower, since loads move 0.72 of the bytes, and its p99 higher (26.8 vs 18.6 s; in both arms the p99 rests on the six slowest of about 580 requests, all 97K–232K-token prompts). Below that, the codec's results follow DRAM plus NVMe in raw-equivalent bytes (Fig. 6a). Tier mode on 400 and 480 GiB of NVMe (1229 raw-equivalent

TABLE III

AGENTX, S = 8, 400 GIB DRAM; P = 64 UNLESS STATED. INPUT THROUGHPUT, ENGINE TTFT, ITL (MS PER OUTPUT TOKEN), THE SHARE OF PROMPT TOKENS SERVED FROM A CACHE (VLLM'S GPU PREFIX CACHE OR LMCACHE) AND GOODPUT (REQUESTS PER MINUTE WITH ENGINE TTFT ≤2 S AND ITL ≤40 MS), OVER THE PROFILING WINDOW. RAW CONTROLS SCALE NVME ALONE OR BOTH TIERS BY 1/0.716, OR MOVE 320 GIB FROM NVME TO DRAM; DRAM ABOVE 400 GIB EXCEEDS THE HARNESS'S CAP (§VIII-C). P99: NEAREST RANK; RATIOS IN THE TEXT USE THE PRINTED CELLS.

			TTFT (s)					
NVMe	arm	k tok/s	mean	p50	p99	ITL	hit	good
2 TiB	raw	27.5	1.69	0.99	18.6	26.0	.94	15.1
	tier mode	26.9	1.85	0.87	26.8	26.8	.94	15.4
	pool mode	28.8	1.34	0.96	5.3	25.6	.94	16.2
1200 GiB	raw	26.1	1.97	1.03	17.3	26.6	.93	14.0
800 GiB	raw	20.0	3.78	1.13	52.7	32.6	.84	9.4
	tier mode	25.0	1.98	0.90	23.8	26.5	.92	14.4
640 GiB	raw	12.2	6.76	1.11	78.7	43.5	.55	4.6
	tier mode	24.6	2.34	0.88	38.0	26.9	.91	13.5
	pool mode	25.5	1.76	0.93	18.8	27.0	.90	13.4
	FP8 serializer	26.9	2.28	1.32	18.4	26.4	.93	12.7
	FP8 KV, raw	33.0	1.06	0.72	6.6	21.6	.94	20.6
	FP8 KV, tier	33.1	1.02	0.70	4.8	21.7	.94	20.6
480 GiB	raw	11.1	7.66	1.45	73.0	53.9	.44	3.5
	tier mode	21.3	3.44	0.97	43.8	31.3	.85	10.5
400 GiB	tier mode	12.6	6.67	1.16	72.0	50.2	.58	4.4
DRAM only, P = 40								
	raw	11.3	7.33	1.91	48.4	62.0	.45	2.9
	tier mode	25.1	1.56	0.80	15.7	30.7	.91	12.5
	FP8 KV, raw	34.1	1.17	0.76	8.2	22.6	.94	19.2
DRAM only, FP8 KV, P = 64								
	raw	16.3	4.32	0.79	37.7	36.5	.61	6.7
	tier mode	30.3	1.33	0.68	13.2	21.7	.90	18.2
Raw controls, DRAM + NVMe GiB								
400+672	raw	14.9	5.86	1.31	73.8	45.2	.66	5.1
559+672	raw	20.9	3.38	0.85	45.4	32.2	.85	10.7
720+480	raw	20.3	3.20	0.84	39.8	33.9	.82	9.9

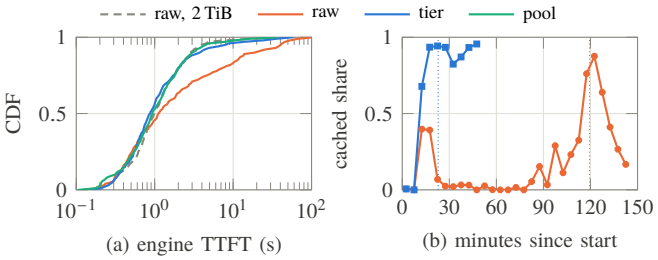


(a) DRAM + NVMe, raw-equiv. GiB (b) sessions, DRAM only

Fig. 6. Compressed tiers act as raw tiers 1/0.716 their size (AgentX, 400 GiB DRAM). (a) 64 sessions, goodput by DRAM plus NVMe over the stored ratio; hollow diamonds: raw controls that scale or move tier bytes (Table III); shaded: the cliff; past the break: 2 TiB. (b) DRAM only, cached share of prompt tokens by live sessions; solid: Qwen3-30B-A3B, dashed: GLM-4-9B.

GiB) matches a raw control with 559 and 672 (1231): 21.3 vs 20.9 k tokens/s, 0.85 cached each, 10.5 vs 10.7 requests per minute. So the same results need less hardware: tier mode on 480 gives raw's goodput on 800 (10.5 vs 9.4) with 0.73 of the DRAM plus NVMe, and on 640 gives raw's on 1200 (13.5 vs 14.0) with 0.65 of it. Above the cliff the curves meet: at 1676 raw-equivalent GiB tier mode has raw's goodput at 1600 (14.4 vs 14.0) but 4% less throughput (25.0 vs 26.1 k), its codec cost and run spread.

On fixed tiers. The same equivalence moves a fixed deployment along a steep curve. With 640 GiB of NVMe, raw



(a) engine TTFT (s) (b) minutes since start

Fig. 7. AgentX, 400 GiB DRAM + 640 GiB NVMe. (a) Engine TTFT over the profiling window; raw LMCache on 2 TiB for reference. (b) Cached share of prompt tokens by 5-minute bin of request completion, until each run ends; dotted lines mark where each arm's profiling window starts. CDFs use the harness's request records and interpolate, so tails differ from Table III's.

LMCache falls off the cliff of Fig. 1: each session's prefix is evicted shortly before the session returns, so the engine recomputes 0.45 of prompt tokens, one 8K-token chunk at a time, and recomputed chunks in every batch raise ITL to 43.5 ms. Its TTFT splits in two (Fig. 7a): half the requests start within 1.1 s, as on 2 TiB, while the slowest quarter, which recompute, take 4.6 s or more (p99 79 s). It caches little for most of its 119 minutes of warmup (Fig. 7b). With the same tiers holding compressed records, 0.91 of prompt tokens stay cached; tier mode has 2.8–2.9× raw's goodput (3.0–3.4× on 480 GiB), p90 TTFT 3.8 instead of 18.9 s and 1.85–2.0× its input tokens (1.8–1.9× its requests), and warms up in 23 minutes. Away from the cliff the gain is smaller (tier mode on 640 against raw on 800: 1.08–1.23× the tokens); on 400 tier mode itself falls off.

More sessions on the same DRAM. Without NVMe, the cliff sits at the DRAM tier, of which LRU keeps 360 GiB at its 0.9 watermark, and the number of live sessions P sets the working set (Fig. 6b). Raw LMCache holds 32 sessions (306 GiB live mean) and falls by 36; tier mode holds 44 and falls at 48. At 36 sessions tier mode serves 3.5× raw's goodput (16.0 vs 4.6 per minute, mean TTFT 1.40 vs 4.81 s), at 40 4.3× (12.5 vs 2.9). GLM-4-9B, a dense model of another family with 0.83 of the KV per token, repeats the step: raw holds 40 sessions (0.88 cached) and falls by 44 (0.46); at 48 tier mode caches 0.92 and serves 7.9× raw's goodput (15.0 vs 1.9 per minute, mean TTFT 1.34 vs 7.38 s), and still 0.87 at 52; it falls by 56. In steps of four sessions the cliff moves by 1.2–1.5× (Qwen) and 1.2–1.4× (GLM), around the stored ratio's 1.4×; the multipliers above are measured between the two cliffs. With vLLM's FP8 KV raw holds 56 (0.90) and falls at 64 (0.61–0.75 cached, 6.6–10.6 per minute), where tier mode caches 0.90–0.91 (17.9–18.5): FP8 records (0.90) move the cliff by only 1.11×, yet past the next step of P.

Cold and warm passes. When every request hits, compression decides which tier serves the hit. The repeat trace sends 48 prompts of 8K, 32K, 64K and 128K tokens (12 each, 64 output tokens, one in flight) and then the same 48 again, so every second-pass request finds its whole prompt in LMCache (cached fraction 1.000 in all arms; Table IV). With 400 GiB of DRAM every hit loads from L_1; the raw

TABLE IV

EVERY REQUEST HITS THE CACHE (REPEAT TRACE, 261 GIB OF RAW KV, 188 GIB COMPRESSED, ONE REQUEST IN FLIGHT): MEAN WARM TTFT WITH RAW LMCACHE TIERS AND ITS RATIO TO EACH OTHER ARM'S (ABOVE 1: FASTER THAN RAW), BY DRAM SIZE AND PROMPT LENGTH. COLD TTFT (RAW, 400 GIB): 0.29, 1.48, 3.90 AND 12.2 S.

L_1		8K	32K	64K	128K
400 GiB	raw (ms)	58	125	211	376
	tier mode	1.00×	1.06×	1.11×	1.05×
	pool mode	1.00×	1.01×	1.07×	1.08×
220 GiB	raw (ms)	93	190	318	586
	tier mode	1.63×	1.64×	1.68×	1.67×
	pool mode	1.67×	1.61×	1.63×	1.70×
160 GiB	raw (ms)	108	219	405	739
	tier mode	1.35×	1.26×	1.41×	1.42×
	pool mode	1.42×	1.17×	1.51×	1.60×
	FP8 serializer	0.88×	0.68×	0.74×	0.73×

load is fast (warm TTFT 58 ms at 8K, 376 ms at 128K), and compression changes warm TTFT by 0–11%. With 220 GiB the compressed set fits in DRAM but the raw set does not: part of the raw hits load from NVMe at the file adapter's 6.6–7.4 GB/s (median warm TTFT stays near the DRAM value, the mean rises 1.5–1.6×), and compression cuts mean warm TTFT 1.61–1.70×. With 160 GiB, below both sets, LRU over the repeated order evicts each record before its reuse and every hit loads from NVMe; reading 28% fewer bytes cuts warm TTFT 1.26–1.42× in tier mode and 1.17–1.60× in pool mode, which copies compressed blobs into the pool without expanding them (its 32K mean varies by 0.14× between runs, as eviction timing changes which hits reach NVMe). LMCache's FP8 serializer reads half the bytes from NVMe but casts them back to BF16 on the CPU (23 GB/s on LMCache's default single thread; a thread pool was slower), and its warm TTFT is 1.14–1.47× raw LMCache's. Cold TTFT stays within 2% of raw LMCache, except one 8K request that directly follows a 128K one, which waits 0.2 s longer in tier mode in every run.

Lossless versus lossy. LMCache's FP8 serializer (stock single worker) stores half the NVMe bytes (DRAM stays raw) and on 640 GiB serves 9% more input tokens than tier mode, with 50% higher median TTFT, because each load is cast back to BF16 on the CPU, and lower p99 (18.4 vs 38.0 s); it changes outputs (§VIII-E). vLLM's FP8 KV halves every level, the GPU pool included: on 640 GiB raw LMCache then serves 1.34× tier mode's input tokens (goodput 20.6 vs 13.5) and lowers multi-key accuracy (§VIII-E). The codec composes with it (33.1 k on 640) and pays once FP8 KV fills the tiers, as at $P = 64$ above.

Step 1. A compressed tier acts as a raw tier 1.4× its bytes: the same DRAM and NVMe hold more sessions, or the same sessions need about 0.7 of them, and hits move to faster tiers. The gain is largest at LRU's cliff, where that capacity decides between a hit and a recompute.

D. Q3: Step 2, the GPU pool

Pool mode keeps vLLM's pool in FragPack records, so a KV budget holds 1.31–1.34× the tokens after the backend's 1.3 GiB of buffers. With NVMe holding every working set, every arm serves 0.94 of prompt tokens from a cache, so only the pool binds, and the gain comes from vLLM's queue, not from loads (Table V).

TABLE V

AGENTX WITH NVME HOLDING EVERY WORKING SET ($P = 64$, 400 GIB DRAM, 2 TIB NVME), AT TWO GPU KV BUDGETS PER GPU AND $S = 8$ UNLESS STATED. POOL: KV TOKENS THE ENGINE HOLDS; WAIT: MEAN REQUESTS WAITING IN VLLM; HIT, GOOD: AS IN TABLE III. BOUND: TIER MODE WITH THE RAW POOL ENLARGED TO 37.4 GIB, ABOUT THE COMPRESSED POOL'S TOKEN COUNT.

				TTFT (s)						
KV	arm	pool	k tok/s	mean	p50	p99	ITL	wait	hit	good
54 GiB	raw	1.18M	27.5	1.69	0.99	18.6	26.0	0.3	.94	15.1
	tier	1.18M	26.9	1.85	0.87	26.8	26.8	0.4	.94	15.4
	pool	1.59M	28.8	1.34	0.96	5.3	25.6	0.2	.94	16.2
	raw, S 12	1.18M	28.7	5.16	1.52	43.5	34.3	1.5	.94	10.6
	pool, S 12	1.59M	31.1	1.94	1.07	14.9	35.6	0.4	.94	14.9
	raw, S 16	1.18M	28.5	13.3	9.06	87.3	38.0	4.4	.94	3.7
	pool, S 16	1.59M	32.5	4.62	1.65	45.6	41.6	1.5	.94	6.8
28 GiB	raw	0.61M	engine stalled (see text)							
	tier	0.61M	24.7	9.43	3.25	70.3	22.1	2.6	.94	7.2
	pool	0.80M	27.4	4.91	1.40	51.6	22.4	1.3	.94	11.3
	bound	0.82M	25.7	5.09	1.12	51.5	24.3	1.3	.94	11.5

More requests in flight. At 54 GiB per GPU and $S = 8$ the raw pool runs every admitted request (0.3 waiting); pool mode adds 4.8% throughput, cuts mean TTFT by 21% and p99 from 18.6 to 5.3 s, with 7% higher goodput. Admitting more overruns the raw pool first. At $S = 12$ it runs 10.3 requests with 1.5 waiting and mean TTFT triples (5.16 s); the compressed pool runs 11.4 with 0.4 waiting, as raw at $S = 8$, and serves 13% more input tokens than raw at $S = 8$ at nearly its goodput (14.9 vs 15.1) and mean TTFT (1.94 vs 1.69 s). At $S = 16$ both queue (4.4 and 1.5 waiting) and pool mode's ITL passes the 40 ms target. GLM-4-9B repeats the step one slot count up: its compressed pool (1.90M vs 1.42M tokens) at $S = 16$ has the queue raw has at 12 (0.7 vs 0.5 waiting), the same goodput (14.1 per minute) and mean TTFT (2.39 s), and 18% more input tokens; raw at $S = 16$ waits 2.5 and falls to 6.4 per minute.

A smaller budget. At 28 GiB per GPU, as 26 GiB more weights per GPU would leave, the pool fits fewer of the eight slots' prompts: tier mode loses 8% of its throughput, 2.6 requests wait instead of 0.4, and mean TTFT rises 5× (1.85 to 9.43 s). Pool mode's 0.80M tokens halve the wait (1.3): it serves 10.9% more input tokens than tier mode and 1.57× its goodput (11.3 vs 7.2 per minute), roughly halves mean and median TTFT (4.91 and 1.40 s) and cuts p99 from 70 to 52 s. Raw LMCache stalled at 28 GiB (finished loads, which vLLM's admission does not count, pinned 72–82% of the pool); tier mode did not, so we count this as a limit of small pools, not a codec gain.

Capacity equivalence. The bound row gives tier mode a raw pool of 0.82M tokens (37.4 GiB), slightly more than the compressed pool's 0.80M and more than a real deployment has. It matches pool mode's wait (1.3), goodput (11.5 vs 11.3) and p99 (51.5 vs 51.6 s): the compressed pool acts as a raw pool of its token count. Pool mode serves 4.6–6.6% more

input tokens than the bound, with 3.5% lower mean TTFT and 8% lower ITL (22.4 vs 24.3 ms). In a profile of steady-state pool-mode steps the fused kernel takes 48% of kernel time, so at 1.14–1.17× per call (§VIII-B) FA3 would lengthen a step by at most 6.7–8.2%. Measured directly (random-token prompts, both arms at the power cap, under 0.01% of records spilled), with eight requests decoding, the compressed pool's step is 1.3, 3.5 and 5.6% shorter than FA3's at 8K, 32K and 96K tokens per request (7.73 vs 7.83, 12.55 vs 13.00, 21.68 vs 22.97 ms). With LMCache attached and 8K prefills that are stored, extend an 80K cached prefix, or import 20 k tokens/s (AgentX's rate), the 96K step is 6.3–7.6% shorter. The remainder over the bound also carries tier mode's 2.2% codec cost and the bound's 2% run spread. With both capacities short (640 GiB of NVMe), pool mode serves 3.5% more input tokens than tier mode with 25% lower mean and 51% lower p99 TTFT, at equal goodput (13.4 vs 13.5).

Step 2. A compressed pool acts as a raw pool 1.31–1.34× its budget, and the fused kernel does not pay this back in decode time: it admits 12 long requests with the queue the raw pool has at 8 (16 at 12 on GLM-4-9B) and serves 13–18% more input tokens at raw's goodput (2.7–2.9× lower mean TTFT when both are over-admitted), and the same results need about 0.75 of the GPU KV memory.

E. Q4: Costs and exactness

Capacity to spare. When raw tiers already hold the working set, the codec buys nothing and costs little. With DRAM alone at $P = 24$ (0.94 cached in both arms), tier mode serves 28.5 vs 28.9 k tokens/s (−1.3%), mean TTFT 1.10 vs 1.15 s and the same goodput (18.1 vs 18.0 per minute); with 2 TiB of NVMe it costs 2.2% of throughput (Q2). **Overhead.** Without reuse every request pays the store and none gains from a load. We replay 320 distinct 32K-token prompts (256 output tokens, eight in flight) through LMCache with 400 GiB of DRAM and 2 TiB of NVMe, so each request stores 3 GiB of KV. Tier mode changes the run's time by 0.3% (574 to 576 s), mean and p99 TTFT by under 0.1% and mean ITL by 0.5%; pool mode adds 0.9% to the run, 1.7% to mean TTFT and 0.6% to ITL. A 100K-token request stores about 9.8 GB, which the encoder encodes in about 18 ms on one GPU (Table II), 9 ms per rank at TP 2.

Tier-only: bit-exact. The GPU test suite compares every restored block byte for byte with its source, including raw fallback, partial tiles, fan-out and corrupted records. End to end, every greedy output with compressed tiers equals the output with raw tiers: all 192 repeat-trace requests of the runs with 220 and 160 GiB of DRAM, cold and warm (SHA-1 of the generated text). Warm and cold passes of one arm differ in summation order (a full hit versus a full prefill; 40 of 48 outputs match), at the same rate for both formats. Warm passes with LMCache's FP8 serializer differ from warm raw passes in 5 of 48. On RULER through LMCache (Qwen3-30B-A3B, 40 prompts per task and length, a subset of the setup below, 88% of warm keys from NVMe), cold passes match across arms, and warm FP8 passes differ from warm raw ones in 2 of 120 multi-key and 19 of 120 variable-tracking answers; multi-key moves from 0.99 to 0.975, variable tracking stays at 0.61, within about 1.5 standard errors. The case for exact tiers is operational: an output never depends on which tier served its prefix, so a tier change needs no requalification.

Pool mode: lossless storage, another kernel. Outputs over coded and raw records are bitwise equal, and greedy outputs are identical whether a prefix comes from ExactKV's tiers or the GPU prefix cache. The kernel sums in a different order than FA3 with the same error: against an FP64 reference at 32K and 128K tokens its mean absolute decode error is 3.1e-5, as is FA3's; on FP8 KV it is 3.0e-5 against FA3 FP8's 5.1e-4, since FA3 also quantizes the query to FP8. This is a rounding-order change, as in the cold/warm comparison above. GSM8K [57] (1319 questions, 5-shot) scores 0.885 with the fused kernel, 0.881 with FA3, 0.883 with vLLM's Triton backend and 0.881 with FP8 KV, within one standard error (0.009). On long-context tasks (Qwen3-30B-A3B, 50 prompts per cell at 32K, 64K and 128K tokens), RULER [56] multi-key needles score 0.99 with the fused kernel and FA3 and 0.95 with FP8 KV. Five-hop variable tracking, whose answers often hinge on whether to stop after the first name, scores 0.66 with FA3, 0.62 fused, 0.62 Triton and 0.66 FP8 (standard error 0.04 at 150 prompts; rerunning FA3 repeats all 150 answers); the fused kernel changes 11 of 150 answers, Triton 41 and FP8 63. Operators who must reproduce FA3's bits run tier-only.

IX. Lessons Learned

Measure capacity first. In an earlier tier store of our own, optimizing load latency (readahead, layer-wise loading, prefetch leases) moved summed TTFT by a few percent; in LMCache, the bytes per session on a fixed 640 GiB tier move input-token throughput by 1.85–2.0× (1.8–1.9× in requests, Table III). The compressed GPU pool, too, pays through vLLM's queue, not through prefix hits (§VIII-D). We then moved the codec into an existing tier instead of building a better one.

Deploy the baseline at its best. Our first LMCache runs lazily grew L_1 and evicted at 80% of the grown pool (45% of its size) in steps of 20% of keys, so DRAM stood 25–45% full and raw LMCache looked worse than it is. Eager allocation, a start gate on tier health and a 90% watermark (§VIII-A) raised raw LMCache's input throughput on AgentX from 25.6 to 27.2–27.5 k tokens/s; every comparison in §VIII is against this deployment.

Keep the store off the load's thread, then count launches. LMCache serves one GPU's stores and loads on one thread; our first port encoded and read lengths there, and every load queued behind a synchronization. The store worker (§V-B) removed that wait without touching LMCache's ordering. A hidden wait remained in the load (a two-deep ring of launch metadata made enqueuing 16 batches take 600 ms instead of 4); metadata uploaded once per batch shape (§V-C) removed it, yet tier mode still lost 7% of raw LMCache's throughput. The cause was neither bytes nor waits: the cache server's kernels run in a CUDA context of their own, which the

GPU time-slices against vLLM's, so every launch, however short, stretches the engine's decode steps. Encoding from the engine's blocks (five launches per batch instead of 22) and a pipelined load brought the loss to 2.2%, against 1.1% between runs of raw LMCache. A microbenchmark on an idle GPU shows none of these costs.

X. Discussion and Limitations

Sizing. A tier's capacity in raw chunks is its bytes over the stored ratio, so the codec needs 0.72 of the DRAM and NVMe for the same working set (0.90 for FP8 KV), and DRAM (at its nominal size) plus NVMe in raw-equivalent GiB orders the runs (Fig. 6a). The cliff lies between 1117 and 1200 GiB of the sum, 1.74–1.87× the live mean (641 GiB), while DRAM alone falls near the live mean (write-through puts recent chunks in both tiers). Warmup follows NVMe alone: at about 1200 GiB of the sum, tiers with 800 GiB of NVMe warm up in 27–28 minutes, with 480–672 in 88–99. **When to turn it on.** Tier mode pays when hits are served from NVMe or raw LMCache caches visibly less than with unbounded tiers; pool mode pays when vLLM's waiting queue is nonzero with prefixes cached. Both are counters the engine and LMCache already export.

Limits. Tiers that already hold the working set gain nothing and pay 0–2.2% (§VIII-E); an operator who can size NVMe well above the working set may leave the codec off, though matching tier mode on 640 GiB takes raw LMCache 1200 GiB of NVMe, and undersized raw tiers warm up in 92–119 minutes instead of 23, against the codec's 1.0 GiB of GPU memory per GPU. The codec scales capacity under any eviction policy; a session-aware policy would move the cliff with or without it. If FP8 KV is acceptable it alone holds sessions BF16 needs the codec for.

Scope. We evaluate one engine per node with TP 1 and 2 (over PCIe). Pipeline and expert parallelism keep the per-chunk record format, but we have not run them, nor several engines sharing one cache server; GLM-4-9B covers the DRAM-only tiers and the pool. FP8 gains less (records 0.90 and 0.84 of raw), and on batches decoding makes the fused FP8 kernel 1.30–1.33× FA3 FP8's time (which also quantizes the query); we have not run the FP8 pool end to end on LMCache. Attention records are laid out for GQA; multi-head latent attention (MLA) caches are out of scope.

XI. Related Work

Tiered KV caches. LMCache [1], Mooncake [2], CachedAttention [3], SGLang HiCache [48], Strata [11], HCache [13] and Tutti [12] store raw KV or states; [14], [8], [15] characterize NVMe offloading and PolarKV [7] and AlayaDB [18] report deployments. **Lossy KV compression.** CacheGen [4] (not in LMCache's multiprocess server, so we run its FP8 serializer), KVTC [55], KIVI [22], KVQuant [23], QServe [24] and FP8 KV change outputs. EVICPRESS [39] chooses per context between lossy compression and eviction; Minima-KV [6] mixes lossy FP8 and 3-bit pages in one paged pool; D-Quant [34] entropy-codes quantized KV into fixed-size bitstreams decoded inside attention. **Lossless compression.** SplitZip [5] targets prefill-to-decode transfer (on FP8, 1.14× for E5M2 and none for E4M3), LEXI [36] inter-chiplet links and TRACE [37] CXL hardware; DFloat11 [31], ZipNN [32] and ZipServ [35] compress weights ([17] also FP8 and KV tensors, offline); the concurrent ExANS [44] ANS-codes BF16 exponents for a KV store (Table II), but ANS state cannot be decoded inside attention; OpenZL [38] targets data at rest; MPC [26] and ndzip-gpu [27] compress scientific floats on GPUs into contiguous outputs. Gorilla [40], Chimp [41] and Elf [28] XOR consecutive floats, BUFF [29] splits bounded floats into queryable sub-columns, and ALP [42] codes leading bits with a 3-bit dictionary plus exceptions, like our primary symbols; ExactKV adds a second escape level and, like FastLanes [43], a lane-parallel layout. **Attention over compressed KV.** Quantized-KV kernels (KIVI, FP8 KV in FA3 [45] and vLLM) dequantize inside attention and ZipServ decodes weights inside GEMM. The concurrent ZipFusion [33] decodes BF16 KV losslessly in FlashInfer's [25] decode attention (1.25× faster on an H100) from fixed 12-bit blocks, raw beyond 16 exponents (21% of our K blocks); it has no tiers. **Compressed buffer management.** The compression cache [49], compressed virtual memory [50] and zswap [51] keep a compressed level in front of a slower one; InnoDB [52] stores compressed pages in fixed-size slots; sized for the worst BF16 tier record of our trace (0.84 of raw, 0.90 with margin), they would add 1.11–1.19× capacity instead of the 1/0.716 = 1.40× of exact lengths, and Umbra [53] and LeanStore [54] manage variable-size pages.

XII. Conclusion

ExactKV stores BF16 and FP8 KV losslessly with a shared 3-bit exponent code, inside LMCache's DRAM and NVMe tiers with no change to vLLM, and in vLLM's GPU pool behind a fused decode-attention kernel that is faster than FA3 on BF16 KV at the power cap. On AgentX, with two model families, its compressed capacity serves as raw capacity 1.4× larger, in two steps. In the tiers, the same DRAM and NVMe hold more sessions: 480–640 GiB of NVMe give raw LMCache's goodput on 800–1200, and 2.8–3.4× its goodput on the same tiers, where LRU's cliff decides between a hit and a recompute; when every request hits, a compressed set that fits DRAM cuts warm TTFT 1.6–1.7×. In the GPU, a compressed pool performs as a raw pool of its token count: 1.57× tier mode's goodput at 28 GiB per GPU, and 12 admitted requests at the queue a raw pool has at 8. With capacity to spare it costs 0–2.2%. In tier mode no output depends on the tier that served it; LMCache's lossy FP8 serializer changes 21 of 240 warm RULER answers (scores within 1.5 SE).

Acknowledgment

Use of AI-generated content. Claude Code (Anthropic) and OpenAI Codex drafted text, figure, prototype and benchmark code, and ran experiments under the authors' direction. The authors checked every claim against logged runs and take full responsibility.

References

[1] Y. Liu, Y. Cheng, J. Yao et al., "LMCache: An efficient KV cache layer for enterprise-scale LLM inference," arXiv:2510.09665, 2025.

[2] R. Qin, Z. Li, W. He et al., "Mooncake: A KVCache-centric disaggregated architecture for LLM serving," FAST 2025.

[3] B. Gao, Z. He, P. Sharma et al., "Cost-efficient large language model serving for multi-turn conversations with CachedAttention," USENIX ATC 2024.

[4] Y. Liu et al., "CacheGen: KV cache compression and streaming for fast large language model serving," SIGCOMM 2024.

[5] Y. Guo and S. Joshi, "SplitZip: Ultra fast lossless KV compression for disaggregated LLM serving," arXiv:2605.01708, 2026.

[6] S. Kozyrev and D. Maiboroda, "Minima-KV: Retention-preserving KV cache compression with mixed-format paged attention," arXiv:2608.23834, 2026.

[7] Y. Zhang, X. Yang, H. Chen, F. Li et al., "PolarKV: Tier locally, serve globally—a KV cache over cloud memory and storage," PVLDB 19(12), 2026.

[8] W. Meng, B. Lee and H. Wang, "Understanding bottlenecks for efficiently serving LLM inference with KV offloading," arXiv:2601.19910, 2026.

[9] W. Kwon, Z. Li, S. Zhuang et al., "Efficient memory management for large language model serving with PagedAttention," SOSP 2023.

[10] L. Zheng, L. Yin, Z. Xie et al., "SGLang: Efficient execution of structured language model programs," NeurIPS 2024.

[11] Z. Xie, Z. Xu, M. Zhao, Y. An et al., "Strata: Hierarchical context caching for long context language model serving," arXiv:2508.18572, 2025.

[12] S. Qiu, Y. Hu, X. Wang, W. Zhu et al., "Tutti: Making SSD-backed KV cache practical for long-context LLM serving," arXiv:2605.03375, 2026.

[13] S. Gao, Y. Chen and J. Shu, "Fast state restoration in LLM serving with HCache," EuroSys 2025.

[14] J. Kanichai, T. De Matteis and A. Trivedi, "Building py-kvcache: A performance characterization of external KV caching for vLLM with NVMe SSDs," arXiv:2609.11744, 2026.

[15] Y. He, D. Shi, Y. Dai et al., "Characterizing predictability-latency trade-offs of KV-cache SSD offloading in LMCache for LLM serving systems," arXiv:2610.21249, 2026.

[16] T. Singh, "The KV cache is the new memory wall," arXiv:2609.30854, 2026.

[17] A. Heilper and D. Singer, "Lossless compression of neural network components: Weights, checkpoints, and K/V caches in low-precision formats," arXiv:2508.19263, 2025.

[18] Y. Deng, Z. You, L. Xiang et al., "AlayaDB: The data foundation for efficient and effective long-context LLM inference," arXiv:2504.10326, 2025.

[19] J. Wang, J. Han, X. Wei et al., "KVCache cache in the wild: Characterizing and optimizing KVCache cache at a large cloud provider," USENIX ATC 2025.

[20] SemiAnalysis, "InferenceX AgentX: Methodology and datasets," https://inferencex.semianalysis.com/agentx, 2026.

[21] B. Liu, H. Qiu, Í. Goiri, R. Fonseca et al., "Agentic coding in the wild: Characterizing GitHub Copilot traces at production scale," arXiv:2608.00101, 2026.

[22] Z. Liu, J. Yuan, H. Jin et al., "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," ICML 2024.

[23] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer and A. Gholami, "KVQuant: Towards 10 million context length LLM inference with KV cache quantization," NeurIPS 2024.

[24] Y. Lin, H. Tang, S. Yang et al., "QServe: W4A8KV4 quantization and system co-design for efficient LLM serving," MLSys 2025.

[25] Z. Ye et al., "FlashInfer: Efficient and customizable attention engine for LLM inference serving," MLSys 2025.

[26] A. Yang, H. Mukka, F. Hesaaraki and M. Burtscher, "MPC: A massively parallel compression algorithm for scientific data," IEEE Cluster 2015.

[27] F. Knorr, P. Thoman and T. Fahringer, "ndzip-gpu: Efficient lossless compression of scientific floating-point data on GPUs," SC 2021.

[28] R. Li et al., "Elf: Erasing-based lossless floating-point compression," PVLDB 16(7), 2023.

[29] C. Liu, H. Jiang, J. Paparrizos and A. J. Elmore, "Decomposed bounded floats for fast compression and queries," PVLDB 14(11), 2021.

[30] NVIDIA, "nvCOMP: GPU-accelerated compression and decompression library," https://developer.nvidia.com/nvcomp.

[31] T. Zhang, M. Hariri, S. Zhong et al., "70% size, 100% accuracy: Lossless LLM compression for efficient GPU inference via dynamic-length float," NeurIPS 2025.

[32] M. Hershcovitch, A. Wood, L. Choshen et al., "ZipNN: Lossless compression for AI models," IEEE CLOUD 2025.

[33] Y. Chen, P. Wang and H. Xu, "ZipFusion: Fusing lossless KV decompression with attention for efficient LLM decoding," arXiv:2610.33719, 2026.

[34] Y. Su, H. Liu, G. Yu and J. Zhu, "D-Quant: Driftable entropy coding for KV cache quantization," arXiv:2609.19880, 2026.

[35] R. Fan, X. Yu, X. Pan et al., "ZipServ: Fast and memory-efficient LLM inference with hardware-aware lossless compression," ASPLOS 2026.

[36] M. Sun, A. Kanani, K. Shroff et al., "LEXI: Lossless exponent coding for efficient inter-chiplet communication in hybrid LLMs," arXiv:2603.15589, 2026.

[37] R. Xie, A. U. Haq, Y. Fang, L. Ma et al., "TRACE: Unlocking effective CXL bandwidth via lossless compression and precision scaling," arXiv:2509.03377, 2025.

[38] Y. Collet, N. Terrell, W. F. Handte et al., "OpenZL: Using graphs to compress smaller and faster," ICDE 2026, arXiv:2605.09928.

[39] S. Feng, Y. Liu, H. Li et al., "EVICPRESS: Joint KV-cache compression and eviction for efficient LLM serving," arXiv:2512.14946, 2025.

[40] T. Pelkonen, S. Franklin, J. Teller et al., "Gorilla: A fast, scalable, in-memory time series database," PVLDB 8(12), 2015.

[41] P. Liakos, K. Papakonstantinopoulou and Y. Kotidis, "Chimp: Efficient lossless floating point compression for time series databases," PVLDB 15(11), 2022.

[42] A. Afroozeh, L. X. Kuffo and P. Boncz, "ALP: Adaptive lossless floating-point compression," Proc. ACM Manag. Data (SIGMOD) 1(4), 2023.

[43] A. Afroozeh and P. Boncz, "The FastLanes compression layout: Decoding >100 billion integers per second with scalar code," PVLDB 16(9), 2023.

[44] OpenLake, "ExANS: Achieving 600 GB/s+ decode throughput at 1.51x lossless compression," https://www.theopenlake.com/blog/exans-lossless-gpu-compression-for-bf16-kv-cache, 2026.

[45] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," NeurIPS 2024.

[46] R. Teknium, J. Quesnelle and C. Guang, "Hermes 3 technical report," arXiv:2408.11857, 2024.

[47] SemiAnalysis, "cc-traces-weka-062126: Claude Code agent session traces," https://huggingface.co/datasets/semianalysisai/cc-traces-weka-062126, 2026.

[48] SGLang Team, "SGLang HiCache: Fast hierarchical KV caching with your favorite storage backends," LMSYS blog, https://lmsys.org/blog/2025-09-10-sglang-hicache/, 2025.

[49] F. Douglis, "The compression cache: Using on-line compression to extend physical memory," USENIX Winter Conference, 1993.

[50] P. R. Wilson, S. F. Kaplan and Y. Smaragdakis, "The case for compressed caching in virtual memory systems," USENIX ATC 1999.

[51] Linux kernel documentation, "zswap," https://docs.kernel.org/admin-guide/mm/zswap.html.

[52] Oracle, "InnoDB table and page compression," MySQL 8.0 Reference Manual, https://dev.mysql.com/doc/refman/8.0/en/innodb-compression.html.

[53] T. Neumann and M. J. Freitag, "Umbra: A disk-based system with in-memory performance," CIDR 2020.

[54] V. Leis, M. Haubenschild, A. Kemper and T. Neumann, "LeanStore: In-memory data management beyond main memory," ICDE 2018.

[55] K. Staniszewski and A. Łańcucki, "KV cache transform coding for compact storage in LLM inference," ICLR 2026.

[56] C.-P. Hsieh, S. Sun, S. Kriman et al., "RULER: What's the real context size of your long-context language models?" COLM 2024.

[57] K. Cobbe, V. Kosaraju, M. Bavarian et al., "Training verifiers to solve math word problems," arXiv:2110.14168, 2021.